

ECO5040W · FINANCIAL SOFTWARE ENGINEERING · UCT 2026

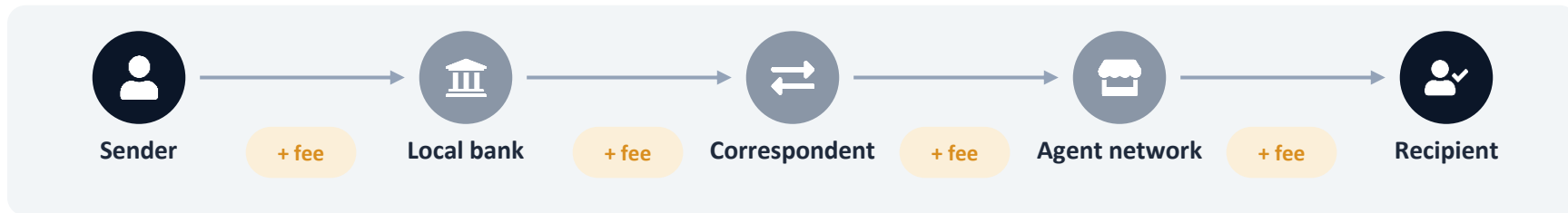
CrossFX

ZAR in, UCTUSD out. A cross-border remittance prototype settling on the XRP Ledger Testnet.

Ndumiso Zondi · Marco Klopper · Muki Mdluli · Rafaela Stevenson



Sending money home costs too much and takes too long



Expensive 6.2%

Correspondent banking and agent networks stack fees. The World Bank puts the global average near 6.2% on a \$200 remittance.



Slow

Value moves through several intermediaries, so senders and recipients wait and cannot trace the money.



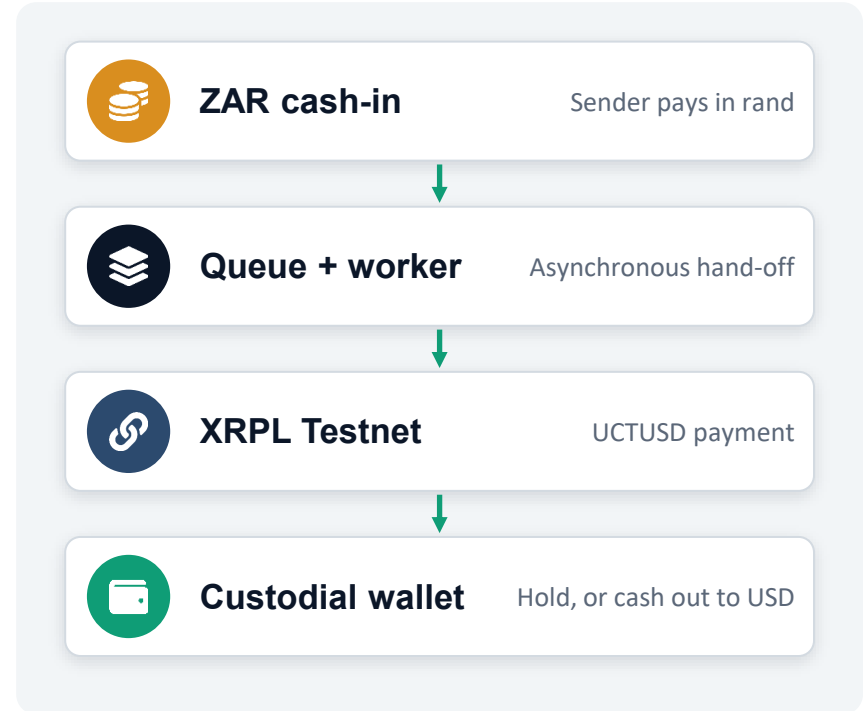
Opaque

The exchange-rate margin is usually buried inside the rate, so the customer cannot see what they paid.

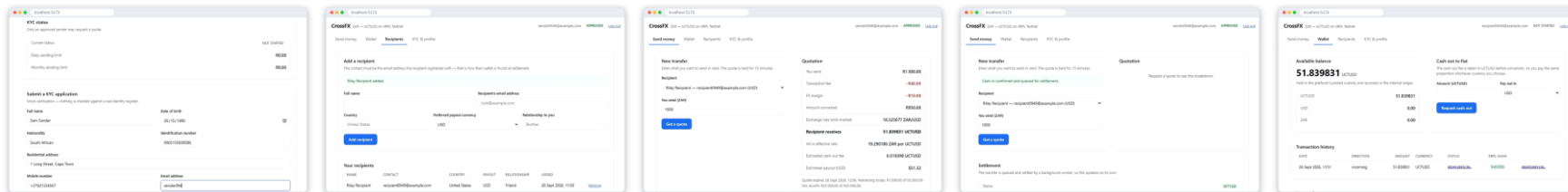
Settle in a stablecoin, disclose every charge

-  A South African sender pays in ZAR; the recipient receives UCTUSD in a custodial web wallet.
-  UCTUSD is the lecturer-issued XRPL Testnet token, standing in for RLUSD as the brief allows. Issuer and currency code are configuration, not code.
-  Settlement is asynchronous: a message queue and a background worker move value on XRPL.
-  The recipient holds the balance or requests a simulated cash-out to USD.

Academic prototype: Testnet only, no real funds.



Register to cash-out in five stages



1

Onboard

Register, log in, complete mock KYC. An admin approves.

2

Recipient

Add a beneficiary. The recipient must be a registered user.

3

Quote

Enter ZAR. See rate, fees, margin, UCTUSD received and estimated payout.

4

Settle

Confirm simulated cash-in. Queue message, worker transfers on XRPL.

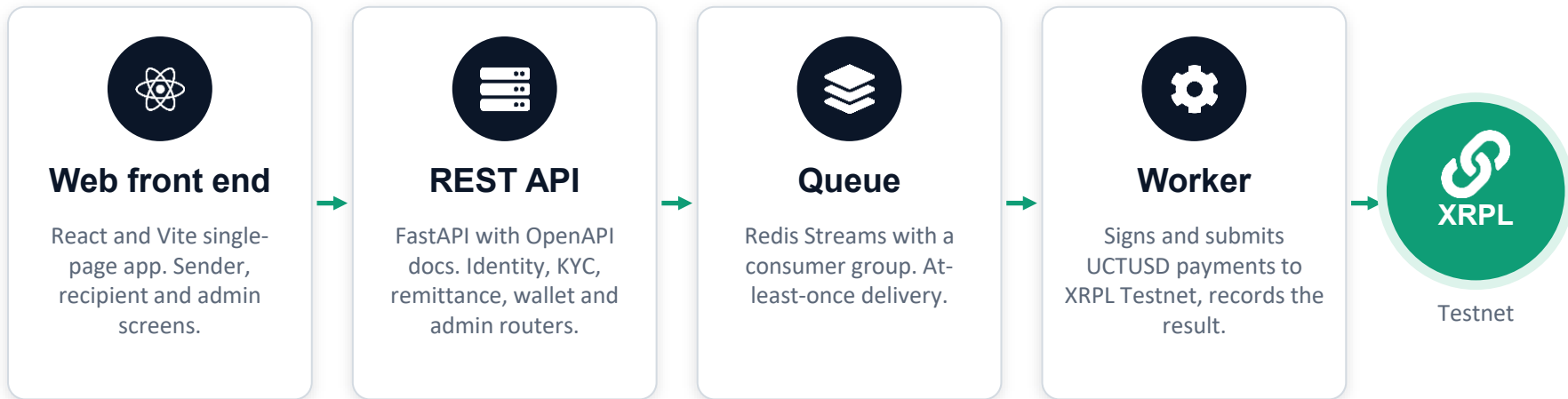
5

Receive

Recipient sees balance and tx hash, then requests cash-out.

Screens captured from our recorded demo run on XRPL Testnet.

An API that hands off, and a worker that settles



Data lives in PostgreSQL (SQLite in dev). Two pooled platform wallets on XRPL hold the liquidity; each user has a claim in an internal ledger.

Every charge is a separate, configurable line

Charge	Default
Fixed fee	R25
Percentage fee	1.50%
FX margin	1.00%
Cash-out fee	1.00%

R1 000 at 18.50	Amount
Fee (R25 + 1.5%)	−R40.00
FX margin (1%)	−R10.00
Converted	R950.00
Recipient gets	51.351351 UCTUSD
Est. USD payout	\$50.83

TOTAL COST

R50

5% all-in, every line shown before paying

All four are environment settings. The margin is charged once; the remainder converts at the mid-market rate.

Confirm cash-in, queue, settle, credit once



Gated

No message is queued until ZAR cash-in is confirmed.



Idempotent

A compare-and-swap claim per remittance means a redelivered message cannot credit twice.



Failures kept

Rejected payments are stored as FAILED with the XRPL result code and no credit.



Reconciled

Payout-pool balance on-chain equals the sum of user claims in the ledger.



Cash-in confirmed



Message queued



Worker claims



Sign and submit



Ledger validates



Hash stored



The TrustSet is done once at setup. Transaction hashes are stored and shown in the recipient wallet.

Live demo: one remittance, end to end

Step	Screen	What we show
1	KYC and profile	Register, submit KYC, edit profile
2	Admin	Approve the KYC application
3	Recipients, Send	Add beneficiary, quote, limit check
4	Send	Confirm cash-in, watch queued to settled
5	Wallet	Balance, tx hash, cash-out request
6	Admin	Approve the payout, queue the burn

Two keys to protect, one module allowed to use them



2

private keys in the whole system, both pooled platform wallets



Encrypted at rest

Pool seeds are Fernet ciphertext. The key sits in the environment, never in the database.



One decrypt point

Only the signing module decrypts a seed. A grep finds a single call.



Never exposed

No route or schema returns a key, and none is logged or committed.



Passwords hashed with bcrypt; login does equal work for unknown users.



JWT auth; admin flag settable only by a local CLI script.



Cross-user reads return 404, not 403.

A stablecoin does not remove the compliance duty



KYC and AML

Identity checks, approval gating and transaction monitoring hooks.



Limits

Configurable daily and monthly caps, R0 until verified.



Exchange control

Cross-border ZAR flows fall under South African capital-flow rules.



Crypto and licensing

Custody, stablecoin rules and payment licences apply to a real service.

Also covered in the spec: protection of customer information, safeguarding of funds and consumer protection. This is not a legal opinion.

TESTING RESULTS

371 backend and 21 frontend tests, run on every pull request



Backend

371

371 pytest tests, ruff clean. Covers auth, KYC, limits, fees, ledger, settlement worker, migrations and security.



Frontend

21

21 Vitest tests, ESLint and a production build.



Every pull request

CI

GitHub Actions runs all of it on every pull request.



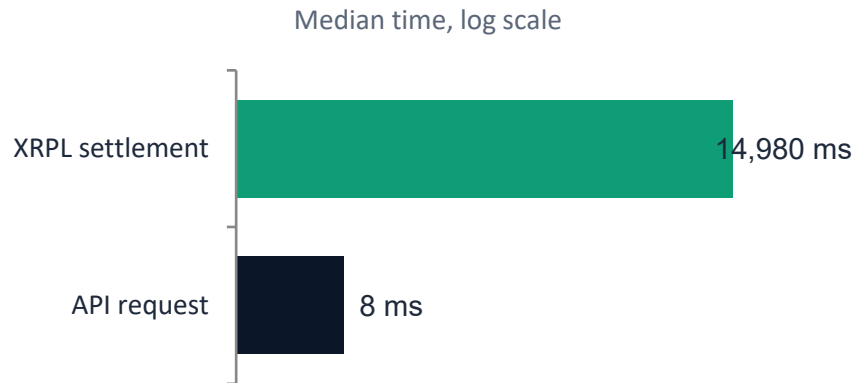
Regression tests pin each fix found in the full audit.



A migration test runs the chain down and up again.

A fast API and a slow ledger, joined by a queue

Measure	Result
API median / p95	8 ms / 22 ms
Sustained throughput	82–84 req/s
HTTP failures	0 of 9,881
Queue publish rate	~27 msg/s
Settlement median	14.98 s
XRPL success (funded)	10 of 10



1,500×

The on-chain leg is about 1,500 times slower than the request that triggers it.

Add workers, not tuning



XRPL round trip

~15 s

About 15 s per settlement, almost all waiting on ledger close. One worker settles 4.4 per minute.



Burst latency

163 s

Ten remittances at once: the tenth waits 163 s, mostly queue time.



bcrypt login

530 ms

530 ms median, by design. Bursts of logins need more API workers.



Not reached: SQLite never became the limit at this load, so the Postgres ceiling is untested. Multi-worker scaling is projected, not measured.

What building it taught us



Audit early

A full audit caught a double-charged margin and a limit check that held on Postgres but not SQLite.



Config, not code

Keeping issuer and currency code in config made the RLUSD to UCTUSD change a settings edit.



Liquidity is real

An unfunded pool fails every settlement with tecPATH_DRY. Nothing warns yet.



Split by contract

Four tracks with agreed API shapes let work run in parallel.

What we would do next



Monitor pool liquidity and refuse quotes the pool cannot settle.



Gate cash-out on recipient KYC and encrypt KYC data at rest.



Match recipients by user id instead of email.



Benchmark on Postgres with multiple API and settlement workers.



Add JWT revocation and refresh tokens.

Thank you. Questions?